Analytical Problem Solving – Assessment Tool 1

Course: Artificial Intelligence (CSA17)

Question 1: Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug (no markings), get exactly 2 gallons into the 4-gallon jug.

State representation: (x, y), where x = water in the 4-gallon jug, y = water in the 3-gallon jug.

Step	Action	State (4-gal, 3-gal)
0	Start	(0, 0)
1	Fill the 3-gallon jug	(0, 3)
2	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour from the 3-gallon jug into the 4-gallon jug until it is full (only 1 gallon fits)	(4, 2)
5	Empty the 4-gallon jug completely	(0, 2)
6	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug	(2, 0)

Result: The 4-gallon jug now contains exactly **2 gallons** of water. The problem is solved in 6 moves.

Question 2: Mars Rover as an Intelligent Agent

i. Percepts

Camera/visual images of terrain, spectrometer/chemical sensor readings, GPS/position coordinates, obstacle-proximity sensor data, temperature and atmospheric readings, wheel odometry data, battery/power level, and commands received from mission control on Earth.

ii. Characterization of the operating environment

- **Partially observable** – sensors have limited range; dust and lighting restrict visibility.
- **Stochastic** – terrain conditions and weather are unpredictable.
- **Sequential** – current actions affect future states (e.g., path already travelled, battery used).
- **Dynamic** – Martian weather/terrain conditions can change while the rover is deciding.
- **Continuous** – position, sensor readings, and movement are continuous-valued.
- **Unknown** – the full terrain map is not known in advance.
- **Single-agent** – the rover generally operates alone (occasional coordination with other rovers/orbiters).

iii. Actions the agent can take

Move forward/backward, turn, drill/dig into soil or rock, collect a sample, take photographs, operate the onboard spectrometer, transmit collected data to Earth, manage solar power/recharge, and re-route to avoid obstacles.

iv. Evaluating agent performance

Distance safely traversed, number and quality of samples collected, accuracy/relevance of scientific data gathered, energy efficiency, time taken to complete assigned tasks, absence of damage or accidents, and total area of terrain explored.

v. Most suitable agent architecture

A **model-based, goal-based agent** (with utility-based elements) is most suitable. Because the environment is partially observable and dynamic, the rover requires an internal model – a map of terrain already seen and known obstacle locations – to reason about parts of the world it cannot currently perceive. Because Earth communication involves significant delay, the rover must plan autonomously toward explicit goals (e.g., “reach coordinate X,” “collect a sample at site Y”) rather than relying on simple reflexes. Utility-based reasoning is also useful for trading off competing factors such as safety, speed, and power consumption when several action sequences could achieve the same goal.

Question 3: 8-Queens Problem – Search Formulation

Goal: Place 8 queens on a chessboard so that no two queens attack each other (no shared row, column, or diagonal).

Problem formulation (incremental approach)

- **States:** Any arrangement of 0 to 8 queens on the board, one per column from the left.
- **Initial state:** An empty board (no queens placed).
- **Actions:** Add a queen to the leftmost empty column, in any row, such that it is not attacked by any queen already placed.
- **Transition model:** Returns the board configuration with the new queen added.
- **Goal test:** All 8 queens are placed on the board and no two queens attack one another (same row, column, or diagonal).
- **Path cost:** Not relevant here – every path that reaches a valid goal state is equally acceptable (constant step cost of 1 per move).

Search space size

A naive complete-state formulation gives roughly 8^8 (~16.7 million) possible boards. Using the incremental formulation above (one queen per column, no two in the same row) reduces the space to only 2,057 possible states, illustrating why careful problem formulation matters.

Suitable search strategies

Backtracking search (depth-first search with constraint checking at each step) is the standard approach. Local search methods such as hill-climbing or the min-conflicts heuristic also work well here, since only reaching a goal state matters and path cost is irrelevant.

Question 4: OLA Cab Booking – Problem-Solving Agent

Agent type: This scenario is best modelled as a **goal-based / utility-based problem-solving agent**. The basic goal is “reach the destination,” but because the customer must choose among several cab options (mini, micro, sedan, shared, prime, etc.) that differ in cost, comfort, and travel time, the agent needs a utility function to rank the options and select the best one rather than simply finding any path to the goal.

Pseudocode

```
function OLA_CAB_AGENT(pickup_location, destination, customer_preferences)
        returns booking

    goal <- REACH(destination)
    cab_types <- [mini, micro, sedan, shared, prime, ...]
    available_options <- []

    for each cab in cab_types:
        distance <- COMPUTE_DISTANCE(pickup_location, destination)
        eta <- ESTIMATE_TIME(cab, distance, traffic_conditions)
        fare <- ESTIMATE_FARE(cab, distance)
        comfort_score <- GET_COMFORT_RATING(cab)
        append (cab, fare, eta, comfort_score) to available_options

    best_option <- SELECT_MAX_UTILITY(available_options, customer_preferences)
        // utility = weighted function of (cost, time, comfort)
        // based on customer preferences

    if best_option is not null:
        CONFIRM_BOOKING(best_option, pickup_location, destination)
        return booking_confirmation
    else:
        return "No cabs available"
```

Question 5: Uniform Cost Search – Logistics Delivery Network

Graph edges (with costs): S→A (1), S→G (12), A→B (3), A→C (1), B→D (3), C→D (1), C→G (2), D→G (3)

UCS trace

At each step, expand the frontier node with the lowest cumulative path cost:

Step	Node expanded	Frontier after expansion (node : cumulative cost)
1	S (0)	A : 1, G : 12
2	A (1)	C : 2, B : 4, G : 12
3	C (2)	D : 3, B : 4, G : 4 (updated from 12, via S-A-C-G)
4	D (3)	B : 4, G : 4 (path via D would give cost 6 — discarded, worse)
5	G (4)	Goal reached — search terminates

Least-cost path: S → A → C → G

Total cost: 1 + 1 + 2 = **4**

Interpretation

For comparison, the alternative routes cost more: S-A-C-D-G = 6, S-A-B-D-G = 10, and the direct edge S-G = 12. The Uniform Cost Search algorithm correctly identifies **S → A → C → G with total cost 4** as the cheapest path from the starting warehouse to the destination warehouse, since it always expands the lowest-cost node in the frontier first and updates costs when a cheaper path to an already-seen node is found.